



Introduction to Runes

Q33

SVELTE 5 RUNES ARCHITECTURE

INTERVIEW QUESTION

Question. What are Svelte 5 Runes, and what specific architectural limitations of legacy reactivity do they address?

ANSWER. Svelte 5 Runes are explicitly declared, fine-grained reactive primitives (such as `$state`, `$derived`, and `$effect`). They replace the legacy component-level reactivity system to address three major architectural limitations: 1. **UNIVERSAL STATE:** Legacy reactivity was strictly bound to `.svelte` files. Runes can be declared in raw `.svelte.js` or `.ts` files, making it effortless to share state across the entire application without using stores. 2. **EXPLICIT DEEP REACTIVITY:** Legacy Svelte relied on the assignment operator (`=`) to trigger updates, which caused confusion when mutating nested arrays or objects. Runes use an explicit Proxy-based signal system, so deep object mutations trigger updates automatically. 3. **FINE-GRAINED DOM UPDATES:** Legacy reactivity dirty-checked the entire component on every state change. Runes use a signal graph to pinpoint exact DOM nodes that depend on the state, bypassing component-level checks and vastly improving performance.

shared.svelte.js

```
// With Runes, reactive state can finally live outside components!
export function createCounter() {
  let count = $state(0);

  return {
    get count() { return count; },
    increment() { count += 1; }
  };
}
```

i Summary. Runes untangle reactivity from the `.svelte` component boundary. They offer explicit, deeply reactive, and surgically fine-grained state management that scales gracefully into large codebases.

